Chapter 3 Exam Notes: Memory Management

Modern Operating Systems Study Notes

Contents

1 Scope

Exam focus: Focus on small conceptual questions: address space, swapping, memory management with bitmaps, virtual-memory math, Section 3.4 page replacement, page-fault actions, segmentation concept. Omit after 3.7.3.

2 Basic Memory Concepts

2.1 Address Space

An **address space** is the set of addresses a process can use. It gives each process the illusion of its own private memory.

2.2 Physical vs Virtual Address

- **Virtual address**: address generated by the CPU/program.
- **Physical address**: actual memory address sent to RAM.
- **MMU**: hardware that translates virtual addresses to physical addresses.

3 Swapping

Swapping moves an entire process between main memory and disk.

How it happens:

1. OS selects a process to move out when memory is scarce.
2. Process memory image is written to disk.
3. Its memory region becomes free.
4. Later, OS reads it back into memory, possibly at another physical address.
5. Relocation/protection hardware ensures addresses remain valid.

Problems: disk I/O is slow, memory holes appear, compaction is expensive, and growing data/stack segments need extra space.

4 Managing Free Memory

4.1 Bitmap

Memory is divided into allocation units. A bitmap has one bit per unit.

- Bit 0 can mean free; bit 1 can mean allocated.
- Easy to store compactly.
- Finding a run of k free units may require scanning many bits.

4.2 Linked List

Keep a list of free and allocated segments with base and length.

- **First fit**: take first large enough hole.
- **Next fit**: continue search from last position.
- **Best fit**: take smallest large enough hole.
- **Worst fit**: take largest hole.

5 Virtual Memory and Paging

5.1 Paging

Virtual address space is divided into fixed-size **pages**. Physical memory is divided into equal-size **page frames**. A page table maps pages to frames.

5.2 Page Number and Offset

For page size S:

$$\text{page number} = \left\lfloor \frac{\text{virtual address}}{S} \right\rfloor$$

$$\text{offset} = \text{virtual address} \bmod S$$

Physical address:

$$\text{frame number} \times S + \text{offset}$$

5.3 TLB

A **Translation Lookaside Buffer** caches recent page-table entries. On a TLB hit, translation is fast. On a miss, the page table must be consulted.

5.4 Effective Access Time

If TLB hit ratio is h, TLB lookup time is t, memory access is m:

$$EAT = h(t + m) + (1 - h)(t + 2m)$$

If page faults are included, add page-fault probability times page-fault service cost. Page faults dominate because disk access is much slower than memory.

6 Page Fault: Actions

When a process references a page not in memory:

1. Hardware traps to the OS.
2. OS saves process state.
3. OS verifies the address is legal.
4. OS finds a free frame or selects a victim page.
5. If victim is dirty, write it to disk.
6. Read needed page from disk into the frame.
7. Update page table and TLB.
8. Restart the faulting instruction.

7 3.4 Page Replacement Algorithms

7.1 Optimal

Replace the page that will be used farthest in the future. Not implementable exactly, but gives a benchmark.

7.2 NRU

Uses Referenced and Modified bits. Prefer pages not recently referenced and not modified.

7.3 FIFO

Replace the oldest loaded page. Simple, but can remove heavily used pages.

7.4 Second Chance

FIFO plus referenced bit. If oldest page was referenced, clear bit and move it to the back.

7.5 Clock

Efficient implementation of second chance using a circular list and hand pointer.

7.6 LRU

Replace the least recently used page. Good approximation to locality, but exact implementation is expensive.

7.7 Aging

Software approximation of LRU. Shift counters periodically and insert referenced bit at the high end.

7.8 Working Set

The working set is the set of pages used in the recent past. Keep a process's working set in memory to reduce thrashing.

7.9 WSClock

Combines clock algorithm with working-set age and dirty-page handling.

8 Segmentation

Segmentation divides a program into logical variable-size regions: code, data, stack, tables, etc.

- Supports logical organization and protection per segment.
- Can share segments between processes.
- Suffers from external fragmentation.
- Segment address = segment number + offset.

8.1 Segmentation with Paging

Each segment can be paged. This combines logical segments with fixed-size page frames. Know the concept only up to Section 3.7.3.

9 Math Problem Templates

9.1 Page Table Size

$$\text{number of pages} = \frac{\text{virtual address space size}}{\text{page size}}$$

$$\text{page table bytes} = \text{number of entries} \times \text{entry size}$$

9.2 Internal Fragmentation

For paging, average unused space in the last page is roughly half a page:

$$\text{average internal fragmentation} \approx \frac{\text{page size}}{2}$$

9.3 Page Replacement Trace

For a reference string:

1. Maintain current frames.
2. For each reference, mark hit or fault.
3. On fault and no free frame, apply the algorithm's victim rule.
4. Count total faults.

9.4 Disk Space for Paging

Worst-case backing store for n processes, virtual size v, RAM r:

$$nv - r$$

This is a worst case and usually unrealistic.

10 Likely Questions with Answers

Question

Q1. Define address space

Answer

An address space is the set of addresses a process can use. It gives each process the illusion of private memory. The CPU generates virtual addresses in this space, and the MMU maps them to physical memory.

Question

Q2. Explain swapping

Answer

Swapping moves an entire process from main memory to disk and later brings it back. The OS chooses a process when memory is scarce, writes its memory image to disk, frees its RAM, and reloads it later when it can run again. Swapping increases multiprogramming but is slow because disk I/O is much slower than RAM.

Question

Q3. How is memory managed using bitmaps?

Answer

Memory is divided into fixed-size allocation units. A bitmap stores one bit per allocation unit, for example 0 for free and 1 for allocated. To allocate k units, the OS scans the bitmap for k consecutive free bits and marks them allocated. The bitmap is compact, but searching for a large free run can be expensive.

Question

Q4. How do you compute page number and offset?

Answer

If page size is P bytes and virtual address is A, then

$$\text{page number} = \lfloor A/P \rfloor, \qquad \text{offset} = A \bmod P.$$

If $P = 2^n$, the low n bits are the offset and the remaining high bits are the page number.

Question

Q5. Explain page-fault steps

Answer

The MMU detects that the referenced page is not present and traps to the kernel. The kernel checks whether the address is legal. If illegal, it kills or signals the process. If legal, it finds a free frame or selects a victim page. If the victim is dirty, it writes it to disk. It reads the needed page into memory, updates the page table/TLB, and restarts the faulting instruction.

Question

Q6. Compare FIFO, clock, LRU, and working set

Answer

FIFO removes the oldest loaded page, but it may remove heavily used pages. Clock approximates second chance by using a reference bit and circular pointer. LRU removes the page least recently used, which is good conceptually but expensive to implement exactly. Working-set replacement keeps pages used in the recent past and tries to remove pages outside the active working set.

Question

Q7. Define segmentation and compare it with paging

Answer

Segmentation divides a program into logical variable-size units such as code, data, stack, and procedures. Paging divides memory into fixed-size pages and frames. Segmentation matches the programmer's logical view and supports protection/sharing per segment, but it can cause external fragmentation. Paging avoids external fragmentation but does not preserve logical program units.